PART A - BIG O TABLE

Method	Structure it touches	Big-O	One-line reason
addSong	HashMap(library), double linked list (head, tail)	O(1)	HashMap doesn't allow duplicates, and double linked list put is O(1), also removeSong is also O(1) due to double linked list prev attr
findSong	HashMap(library)	O(1)	`get` hashes the id and jumps straight to its node, so there's no scanning, and `.song` is one field read.
play	HashMap(library, playsCount), Deque(history)	O(1)	The library lookup and the play-count update are O(1) average hash operations, and `push` onto history is O(1).
queueLater	HashMap(library), ArrayDeque(upNext)	O(1)	The library look up are O(1) avg, and .addLast() also O(1)
queueNext	HashMap(library), ArrayDeque(upNext)	O(1)	The library look up are O(1) avg, and .addFirst() also O(1)
next	ArrayDeque(upNext, history), HashMap(library, playsCount), doubly linked list (node `next`),	O(n)	It has a while loop that iterates over the whole upNext queue in the worst scenario. Best case scenario it's O(1) when the front song is valid
back	ArrayDeque(history, upNext), HashMap(library, playsCount)	O(n)	It has a while loop that iterates over the whole history queue in the worst case scenario.
removeSong	Doubly linked list (head, tail, prev and next), HashMap(playsCount, library)	O(1)	Finding and removing a node from a doubly linked list is O(1)

showPlaylist	ArrayList(songs), doubly linked list(head)	O(n)	Doubly linked list traversal is O(n)

PART B: Defend two choices

Doubly linked list (the playlist)
The playlist keeps songs in order, so `next()` can move to the following song with `node.next`, and `removeSong` can unlink any node in O(1) due to each node having `prev` and `next` links.
I built it with my own Node class. Also the library map stores the node itself, so I can find it in O(1). Finding, and removing specific songs would have been O(n) if I used a plain array.

Plays counts (HashMap<String, Integer>)
The play count map records how many times each song has started. `startSong` updates it, and `play`, `next` and `back` all go through that one method. `playCount`(the method not play**s**Count) and `topSongs` read from it. Without a HashMap I would have gone with two parallel arrays that one has ids of the songs played, and the 2nd array holds the number of play counts. Finding and removing from it would have been O(n) and also I would need to be careful not to mess up with the indexes. However, using a HashMap look up and removal is O(1) and I don't have to worry about duplicates and indexes. I reused Java's HashMap, building my own would have meant writing hashing, and resizing, which is too much work.